SWE EXCELLENCE · FIRST DAY ON THE DEVICE

FIELD HANDBOOK

What to do, what to ask and what to check when you build, debug or simply need to understand an application on an RTOS.



Gates between steps:

- After 2: can you state the scheduler policy, tick rate and number of priority levels?
- After 3: does every task have a trigger or period, a priority, a stack size and an owner?
- After 4: does every periodic component have a deadline and an estimated or measured budget?
- After 5: do you have one number per requirement (p99.9, max, drops, misses) — not an average?

Then branch: BUILD → component card · DEBUG → symptom triage · UNDERSTAND → trace one message end to end

A working document: print it, write in it, keep it next to the keyboard.

Contents

How to use this handbook 3

PART 1 · START

1 First hour on the device 5
2 Inventory forms 8

PART 2 · BUILD

3 The Component Card 11
4 Timing budget sheet 14
5 Build checklist: before you merge a component 17

PART 3 · DEBUG

6 Debug triage: symptom to first probe 19

PART 4 · UNDERSTAND

7 Understanding a system you did not write 22

PART 5 · REVIEW

8 Code-review checklist for RTOS changes 24

PART 6 · REFERENCE

9 Measuring recipes 26
10 The implicit-requirements checklist 29
11 Incident patterns 31
12 Quick reference 33

How to use this handbook

Open it when you are handed a device and a task. It does not teach; the book does. It tells you what to do next, what to ask, and what to check, in the order that finds problems earliest.

The three jobs

You are asked to...	Start at	Then	Finish with
Understand an application on a device	§1 First hour	§2 Inventory forms, §7 Trace one message	a filled recon sheet and one traced message
Build a component or feature	§1, §3 Component card	§4 Timing budget, §5 Build checklist	the card, the budget, a passing evidence table
Debug a problem on the device	§6 Triage	§9 Measuring recipes	the first bad state, with evidence, not a guess

The shape of every task

1. **Run it.** Build with debug symbols, run it (hardware or emulator), read the log. Write down what you saw. (The *State First* book's step 1.)
2. **Inventory it.** Tasks, interrupts, queues, shared data, rates. Forms in §2.
3. **State it as numbers.** Period, deadline, budget, depth, owner, margin: the Component Card (§3) and Timing Budget (§4).
4. **Probe it.** Trace, counters, histograms. Predict first, then measure (§9).
5. **Prove it.** One number per requirement, with a method and a margin (book ch. 20).
6. **Log it.** Keep the cards, the traces and the counters with the code.

What to bring

Item	Why
the framework's configuration file(s)	the first document (book ch. 22)
a way to run: board or emulator, and a debugger	step 1
the book's Appendix B (formulas) and this handbook, printed	forms to fill in
`labs/host/rtcalc.py`	schedulability in one command

Item	Why
a pen	forms are meant to be written on

Marks

[book ch. N] points to the explanation. ☐ is a check to tick. A blank line is a question you must answer or hand to someone who can.

> **RULE** **One rule above all.** If you cannot fill in a line, do not guess it. Write `?` and make “find out” a task. A `?` on a form is a finding.

PART 1 · START · SECTION 1

First hour on the device

Five steps, each with a gate. By the end you can say what the scheduler does, what runs when, what is shared, and how to see inside, without having read the whole code base.



Gates between steps:

- After 2: can you state the scheduler policy, tick rate and number of priority levels?
- After 3: does every task have a trigger or period, a priority, a stack size and an owner?
- After 4: does every periodic component have a deadline and an estimated or measured budget?
- After 5: do you have one number per requirement (p99.9, max, drops, misses) — not an average?

Then branch: BUILD → component card · DEBUG → symptom triage · UNDERSTAND → trace one message end to end

Figure 1. The first hour. Do not skip a gate; a “no” is the most useful thing you will learn today.

Step 1 · Run it

☐ Build with debug symbols and **no optimisation** (`-O0 -g`, or `-Og`) so variables are visible. Note the exact command. ☐ Run it on the real target or the emulator. Save the first 200 lines of output. ☐ Run it twice. Do the two logs match? If not, note which lines differ (timing? ordering?). ☐ Write down: **one thing it did that surprised you.** It is your first lead.

Step 2 · Read the configuration

Open the kernel configuration (`FreeRTOSConfig.h`, Zephyr `prj.conf`/Kconfig, the AUTOSAR OS config, the cFS platform config). Fill the **Framework Recon Sheet** on the next page. Anything that says “default” or “I think” is a `?`.

Gate A. *Can you state the scheduler policy, the tick rate, the number of priority levels, whether it preempts, and what detects a stack overflow?*

Step 3 · Inventory

Fill the forms in §2: every task, every interrupt, every queue, every shared datum. Sources: the running system (a task list command or debugger), the framework's topology/launch/schedule files, a search for locks and queue creation.

Gate B. *Does every task have a trigger or period, a priority, a stack size and an owner? Does every queue have a depth, an overflow policy and a drop counter?*

Step 4 · Budget

Fill the Timing Budget (§4) from the inventory: period, deadline, budget, load. Run `rtcalc.py` on it.

Gate C. *Does every periodic component have a deadline and a defended budget? What is the utilization and the worst response of the lowest-priority task?*

Step 5 · Measure

Pick **one** question from the inventory you are least sure of and measure it with the recipes in §9 (a histogram, a counter, a trace).

Gate D. *Do you have one number per requirement (p99.9, max, drops, misses), not an average?*

Then branch

BUILD → §3 Component Card and §5 checklist · **DEBUG** → §6 Triage · **UNDERSTAND** → §7 trace one message.

Framework Recon Sheet

FRAMEWORK RECON SHEET

System / framework: _____ **Version:** _____ **Date:** _____ **By:** _____

Target: board / emulator _____ **CPU:** _____ **Clock:** ____ MHz **RAM:** ____ **Cores:** ___

Scheduler policy (fixed-prio / EDF / cooperative / table)

Preemptive? time slicing?

Tick rate and tick width (16/32/64-bit) → wraps in

Priority levels / idle / timer-task priority

Memory: static or heap? scheme? size

Overflow checks (stack, heap hook, assert)

Mutexes with priority inheritance?

ISR-safe API names / interrupt priority limit

Watchdog: who feeds it, from where

Reset reason / post-mortem available?

Lifecycle: who starts what, in what order

Message system (queues / bus / topics) + QoS defaults

Trace / stats hooks enabled?

Log output (UART / RTT / SWO) and its cost

Where the framework's own docs say the above (file / page)

What surprised me in the first run

? items to resolve (owner · date)

Inventory forms

Four tables describe almost any RTOS application: tasks, interrupts, queues and shared data. Fill them from the running system and the framework's files, not from memory. Photocopy as needed.

How to get each table

Table	Fastest source
Tasks	task-list command; gdb helper (`fr-tasks`); framework topology / launch / schedule files
Interrupts	vector table; peripheral init code; interrupt priority settings
Queues & messages	queue/pipe/topic creation sites; QoS or depth settings; framework port wiring
Shared data	search for mutexes, critical sections, `volatile`, global state; guarded ports
Stack/heap margins	high-water marks after a worst-case run

Task table

TASK TABLE · SYSTEM _____ · DATE _____

Task / component	Trigger (period / event)	T or min gap	D	C budget (how known?)	Prio	Stack (free min)	Blocks on	Owner

Also list: idle task · timer service task (priority!) · any task the framework created for you.

Notes / ?

INTERRUPT TABLE

Interrupt	Source	Priority (above the kernel line?)	Rate / min gap	Handler time (worst)	Does	Uses

Tick interrupt: rate ___ Hz cost ___ µs **Longest interrupts-off section:** ___ µs (where: _____)

Queue and message table

QUEUE / MESSAGE TABLE · ONE ROW PER CONNECTION

From → to	Message (size)	Rate avg / peak	Depth (from burst?)	Full → (policy)	Drop counter?	Age limit (stamped where?)	QoS / ownership

Depth rule: ⌈(peak rate − service rate) × burst length⌉ + 1. **Latency if full:** depth × service time = _____

SHARED DATA TABLE

Datum	Owner (writer)	Readers	Protection (copy / crit.sect / mutex / atomic word)	Longest hold	Blocks whom	Invariant that detects tearing

Lock order (if more than one lock): _____ → _____ → _____ **Locks held across a blocking call?** □ none □ list: _____

PART 2 · BUILD · SECTION 3

The Component Card

One page per component. It is the timing contract, the data contract and the failure contract in one place. Fill it before you write code; update it when the code changes. A reviewer should be able to approve or reject a change from the card alone.

Component Card (blank)

COMPONENT CARD · COMPONENT _____ · OWNER _____ · VERSION/DATE _____

PURPOSE (one sentence)

TRIGGER ☐ periodic ☐ sporadic ☐ message ☐ interrupt

RATE period T / min gap _____ jitter limit _____

DEADLINE D = _____ (source: _____)

BUDGET C = _____ (☐ analysed ☐ measured max ☐ guessed)

PRIORITY _____ (rule: _____) **stack** ___ words, min-free ___

MISS POLICY ☐ continue ☐ skip ☐ abort ☐ degrade ☐ reset

INPUTS (name · from · rate · queue depth · overflow policy · max age · QoS)

OUTPUTS (name · to · rate · stamped where, by what clock)

SHARED DATA touched (datum · read/write · protection · longest hold)

BLOCKS ON (locks, queues, I/O, delays; timeouts)

MEMORY static / heap allocations after init? (should be none) · pools · growth with time

LIFECYCLE start-up needs (who must exist first) · stop/drain · state after restart

FAILURE detector · reaction · blast radius · recovery · evidence kept

EVIDENCE requirement → metric → method → margin → result

? OPEN QUESTIONS (each with who can answer)

Component Card (worked example: the filter in lab R10)

COMPONENT CARD · FILTER · OWNER: PIPELINE TEAM · BASELINE OF LAB R10

PURPOSE Turn each raw IMU sample into an attitude estimate and publish it.

TRIGGER message (one raw sample per ≈997 µs, from the IMU interrupt through `q_raw`). **RATE** ≈1,003 Hz; sensor has its own clock (not harmonic with the 10 ms control).

DEADLINE estimate published ≤ 3 ms after the IMU interrupt for 99.9% of samples (R-2). **BUDGET** 400 µs per sample (measured); **plus a 6 ms recalibration every 250th sample** ← does not fit the rate: a defect.

PRIORITY 4 (above control, below actuator). The logger runs at **6**: wrong (it can delay the filter by its whole 3 ms).

MISS POLICY not defined (the queue silently drops). ← a defect.

INPUTS `q_raw`: depth **4**, overflow = drop newest, **no counter visible**, age limit 3 ms, stamp at the interrupt (hardware timer). ← depth needs ≥ 8 and a visible counter.

OUTPUTS `att` (shared struct: angle, ~angle, seq, t_us), stamped with the sample's interrupt time.

SHARED DATA `att`: writer = filter; reader = control; protection = **none**; two reads at different times → torn reads (47 of 309 cycles). ← needs a snapshot.

FAILURE detector: none · reaction: none · blast radius: control gets stale or mixed data · evidence: none.

EVIDENCE R-1 (no loss): `dropped_raw` = 54 → fail · R-2: p99.9 = 9.5 ms → fail · R-3: `torn` = 47 → fail · R-4: `ctrl_late` = 61 → fail.

AFTER FIXES logger priority 1 · depth 8 + counter · snapshot under a critical section · recalibration moved to a priority-2 task → all four met (p99.9 = 631 µs).

PART 2 · BUILD · SECTION 4

Timing budget sheet

The budget is where the card's numbers meet arithmetic. It answers "can my component finish on time, given everything above it?" with a number, a slack and a verdict. Do it before you build, and again after you measure.

Procedure

1. **List everything that takes CPU**: every task, **every interrupt (including the tick)**, the timer-service task, framework tasks.
2. **Assign priorities** (rate-monotonic unless the framework has fixed them). Record any that violate the rule and why.
3. **Write C, T, D** for each. Use the *measured maximum plus margin* for C, and call it "claimed".
4. **Utilization** $U = \Sigma\, C/T$. Compare with the Liu–Layland bound for n (book ch. 8). If above, continue.
5. **Response time** for each task: $R = C + B + \Sigma\, \lceil (R + J)/T \rceil \cdot C$ of higher priority (ch. 9). Use `rtcalc.py`.
6. **Blocking B**: the longest critical section of any lower-priority task on a lock your task or a higher one uses (ch. 13).
7. **Overhead**: add ≈ a few tens of µs per wake-up and per tick (measure on your target).
8. **Slack** = D − R. **Require slack ≥ 10–20% of D** (a team choice; zero slack misses in practice: book ch. 9).
9. **Compare with a measurement** (§9). Disagreement of more than the overhead means an input is wrong.

Calculator

```
python3 labs/host/rtcalc.py name:C:T[:D[:B]] name:C:T ...   [--edf] [--gantt 100]
python3 labs/host/rtcalc.py isr:250:2000 t1:1000:4000 t2:2000:6000 t3:3000:12000
```

Put the interrupts in as tasks. Same time unit throughout.

Timing budget sheet

TIMING BUDGET · SYSTEM _____ · CPU _____ MHZ · TICK ____ HZ · DATE _____

Task / ISR	Prio	T	D	C (claimed)	U = C/T	B	J	R (worst)	Slack D–R	OK?	Measured max

Total U = _____ **n** = ___ **LL bound** = _____

Harmonic periods? □ yes □ no · hyperperiod _____

Overhead assumed (tick, switch): _____ µs per _____

Tasks with slack < 10% D:

Critical section list (lock · holder priority · longest hold · who it blocks)

Verdict & actions

Reading the result

Finding	Likely cause	First action
R > D for one low-priority task	too much higher-priority load, or a large B	find the biggest contributor term; shorten critical sections; move work down
R > D for everyone	U too high; or an interrupt is missing from the list	list interrupts; measure; consider rates
R ≤ D on paper, miss on target	zero slack plus overhead; an unlisted task; C underestimated	add overhead; list framework tasks; re-measure C under load
measured ≪ predicted	not at the critical instant; C overestimated	do not rely on it; the test never aligned the tasks
U ≤ bound but a deadline is missed	blocking, jitter, or D < T	use RTA with B and J

PART 2 · BUILD · SECTION 5

Build checklist: before you merge a component

Twenty-six checks, grouped by the six questions. Each takes a minute if you filled in the card and budget; each is a gap that has caused a real failure if you did not. Tick, or write the owner who will answer.

1 · Timing

☐ T1 Trigger, period/min gap, deadline and budget written on the card. ☐ T2 Budget is an **upper bound** with margin, and says how it was obtained. ☐ T3 Miss policy defined (what happens to the *next* release). ☐ T4 Every periodic wait uses an **absolute** time (`DelayUntil`), initialised correctly (not in the future). ☐ T5 All counters/timeouts are compared by **difference**; wrap times known (tick, µs timer). ☐ T6 Interrupts and tick are in the budget.

2 · Scheduling

☐ S1 Priority follows the rate/deadline rule, or the exception is documented. ☐ S2 The task **blocks** somewhere (no polling spin); nothing blocks without a timeout. ☐ S3 Timer callbacks do no blocking and take microseconds. ☐ S4 Response time $R \leq D$ with slack, computed or measured.

3 · Messaging

☐ M1 Every queue: depth justified from a burst; overflow policy named; **drop counter exists and is read**. ☐ M2 Every message carries a **source timestamp** and, if order matters, a sequence number. ☐ M3 Age limit defined and checked at the consumer. ☐ M4 Start-up order: consumers exist and are ready before producers run. ☐ M5 QoS (if any) written down per subscription.

4 · Synchronization

☐ Y1 Every shared datum: owner, protection, longest hold (card). ☐ Y2 Multi-field data read as one snapshot (or passed by value); invariant to detect tearing. ☐ Y3 Mutex (with inheritance) for exclusion; no binary semaphore as a lock. ☐ Y4 No blocking call while holding a lock; lock order documented. ☐ Y5 Interrupt handlers use only ISR-safe calls and no locks.

5 · Reliability

□ R1 If the component hangs: who notices (check-in, deadline monitor), how fast, what happens. □ R2 Stack sized from a measured high-water mark with margin; overflow detection on. □ R3 No allocation after initialization (or failure handled and bounded). □ R4 Restart behaviour defined (state, repeated failure, boot loop). □ R5 Evidence survives a reset (reset reason, who was missing).

6 · Observability

□ O1 Each requirement has a metric, a method and a margin written down. □ O2 Latency reported as a histogram; open-loop; p99.9 and max. □ O3 Releases, completions and misses counted separately. □ O4 A trace hook or ring buffer exists and can be dumped on error.

Sign-off

Reviewer _____ date _____ open `?` items: _____ **Merge only when every box is ticked or has an owner and a date.**

PART 3 · DEBUG · SECTION 6

Debug triage: symptom to first probe

On a device the debugger is a blunt instrument: stopping the CPU changes the timing you are investigating. Start from the symptom, name the likely class of cause, and pick the probe that observes without disturbing. The method is the one in *State First*: state the data and conditions, predict, then probe.

Symptom	First suspicion	First probe
Missed deadline / jitter	overrun, preemption, ISR load, inversion	**trace the window; compare R to budget**
System alive, one function dead	task blocked forever (lock, queue)	**list tasks + what each waits on**
Resets / reboot loop	watchdog, fault, boot-time resource	**read reset reason + saved post-mortem**
Wrong or inconsistent values	race on shared state, stale data	**invariant check; watch the variable**
Messages missing / late	queue full, consumer starved, QoS	**drop counters, queue depth, age stamps**
Slowly gets worse over hours	leak, fragmentation, counter wrap, drift	**min-ever-free heap, high-water marks, uptime**
Works in debug, fails in release	timing/optimisation, uninitialised data	**trace instead of breakpoints; build flags diff**

Figure 2. One row per symptom: the first suspicion and the first probe. Details for each follow.

The rules of probing on a real-time system

Rule	Reason
Predict before you probe: write the number you expect	a probe without a prediction teaches nothing
Counters and traces before breakpoints	a stop costs time (in our emulator about 1 ms each) and may pause watchdogs
One change at a time, same test, same load	timing bugs move when you touch them
Reproduce first (make it deterministic if you can: fixed load, emulator with instruction counting)	a bug you cannot repeat you cannot fix
Look at the tail, not the average	requirements are worst-case
Keep the evidence (trace, counters, build id)	the next person needs it

Playbooks

A · A deadline is missed, or jitter is high

1. Which task, which job, by how much? (histogram of response time; count releases vs completions)
2. Trace the window around the worst job: who ran, who was blocked on what, which interrupts fired.
3. Compare with the **budget**: is C larger than claimed (slow path)? Is something higher-priority running longer (a logger, a timer callback)? Is it **blocked** on a lock held by a lower-priority task (inversion)?
4. Fix class: shorten the slow path; move work down in priority; bound or shorten critical sections; use a mutex with inheritance; add margin. Re-measure.

B · The system is alive but one function is dead

1. List tasks with state and what each waits on (`fr-tasks`, framework task list).
2. A task "blocked forever" on a lock or queue is the usual cause. Who holds the lock? Is the producer alive? (A binary semaphore has no owner: look at who *should* have given it.)
3. Check the watchdog design: would it notice this? (book ch. 19)

C · Resets or a boot loop

1. Read the **reset reason**. Watchdog, brown-out, fault, software?
2. Read the saved evidence (`.noinit` marker, check-in mask, trace tail, fault registers: stacked PC/LR).
3. If it resets at start-up every time: what grows with time in service (files, logs, counters)? Is a resource consumed at boot? (Spirit)
4. Make the evidence persist before the reset in the next build.

D · Wrong or inconsistent values

1. State an **invariant** the data should satisfy (field and complement; sequence read twice; checksum).
2. Count violations; correlate with other activity (which task or interrupt runs at that instant?).
3. Put a **hardware watchpoint** on the variable (condition on the bad state); on a hit, inspect which task is running and the call stack.
4. Fix class: snapshot, ownership, critical section; never `volatile` alone.

E · Messages missing, late or duplicated

1. Counters at each hop: produced, enqueued, dropped, dequeued, processed.
2. Queue depth high-water; oldest-item age. Is the consumer starved or blocked?
3. Check start-up order: did the producer run before the consumer existed?
4. Check QoS (reliable? depth? lifespan?) and timestamps (assigned where?).

F · Gets worse over hours or days

1. Heap: minimum-ever-free and largest free block over time. Stack high-water marks. Queue depths.
2. Counter widths and wrap times (tick, µs timers, sequence numbers). Uptime arithmetic.

3. Clock drift: anything multiplied by uptime? (Patriot)
4. Logs, files, caches that grow.

G · Works in the debugger or debug build, fails in release (or the reverse)

1. Timing changed: compare optimisation levels and the cost of logging. Is the failure a race?
2. Uninitialised variables (locals are *undefined*), stack sizes differing, `volatile` missing.
3. Observe with traces and counters in **both** builds.

H · CPU load is too high

1. Per-task CPU share (run-time stats); interrupts' share; idle share.
2. Look for a spin (a task that never blocks), a timer callback that is long, a logger at high priority, a retry storm.
3. Re-run the budget (§4) with measured C.

Using gdb without wrecking the timing

Goal	Recipe (see §9)
log without stopping	`break FUNC if cond` + `silent` + `printf` + `continue`, **only on rarely-called functions**
stop on a bad state	`watch VAR if VAR == BAD` (hardware watchpoint)
see all tasks	helper `fr-tasks` (source `labs/rtos/gdb/freertos.py`)
find who runs	`fr-tick`; `print pxCurrentTCB->pcTaskName`
understand a lock	`fr-queue LOCK` (owner? waiters?)

The debugger is not neutral. In our emulator two silent breakpoint stops moved the clock by about 2 ms, and a breakpoint on a function called in the first cycle made the first deadline miss disappear. On hardware, timers keep running while halted and some watchdogs stop.

PART 4 · UNDERSTAND · SECTION 7

Understanding a system you did not write

The goal is not to read all the code. It is to trace **one message** from where it is born to where it takes effect, and to know every wait, queue, lock and clock on the way. After that, the rest of the system is a set of neighbouring traces.

Trace one message

Pick the most important output of the system (a command to a motor, a value on a display, a telemetry packet). Then walk backwards and fill this chain for each hop.

Hop	Question	Look for
Sink	where does it take effect, and by when must it?	the deadline / requirement
Producer of the sink's input	which task, at what rate and priority, produces it?	task table row
Queue / shared data	how does it get from producer to consumer? depth, policy, protection	queue and shared-data rows
Earlier stages	repeat until you reach…	
Source	the interrupt or sensor; its clock; where is it stamped?	interrupt table row; timestamp site

Then compute: **worst-case data age** at the sink = Σ (T + R) along the chain (book ch. 17) and compare with the requirement. If the framework hides a hop (a middleware, an executor), find it in the documentation and write it in.

Reading order for a code base

1. **Configuration** (kernel config; framework topology/launch/schedule). It lists the threads, rates, queues.
2. **Start-up code**: what is created, in what order, what is enabled last.
3. **Interrupt handlers**: short? what do they post?
4. **The tasks**: each loop: what does it wait on, what does it do, what does it post?
5. **Shared state**: grep for locks, critical sections, `volatile`, globals written by more than one flow.
6. **Error paths**: assert, hook functions, fault handler, watchdog feeding, reset.
7. **Logging and trace**: what exists to observe the system.

Questions to ask the people (and the framework docs)

Topic	Ask
Rates	Why is this component at this rate? What would break at half? At double?
Deadlines	What is the real requirement behind each deadline, and who owns it?
Budgets	Where do the execution-time numbers come from? Worst case, or typical? Measured on which hardware?
Priorities	Who assigned them and by what rule? Has anyone re-analysed since the last change?
Queues	What happens when this queue is full? Who would notice?
Shared data	Who else writes this? What stops a torn read?
Start-up	What must exist before this starts? What enforces the order?
Failure	What is the watchdog fed by? What does the system do on the third reset in a row?
Observability	What do you look at when this misbehaves? Is there a trace? A histogram?
History	What was the last timing bug here, and what changed afterwards?

Done when…

☐ You can name every task, its rate and priority from memory. ☐ You can trace the main message end to end with its worst-case age. ☐ You know what happens when each queue is full and when each task hangs. ☐ You know one way to see inside the running system. ☐ Your `?` list has owners.

Code-review checklist for RTOS changes

What to look at in a diff that touches tasks, interrupts, queues, locks or timing. Each item is a failure that has happened. Review the diff against the Component Card, not against taste.

Interrupts and timing

☐	Check
☐	New or changed ISR: only ISR-safe calls; no blocking; no locks; no print; short (state its worst case).
☐	Anything done with interrupts disabled: how many microseconds? Any loop in it?
☐	A delay or timeout: absolute time for periods; difference compare for wrap; `last` initialised correctly.
☐	A busy-wait or `for(;;)` without a blocking call.
☐	Anything that takes a variable time (float, division, search, printf, flash/EEPROM write) in a high-priority path.
☐	A timer callback: does it block or take long?

Tasks and priorities

☐	Check
☐	New task: stack size justified (high-water mark), priority justified (rule), name set, created in the right order.
☐	Changed priority: re-ran the budget? Any equal-priority neighbours?
☐	Task deletion: who frees what; is the idle task able to run?
☐	Anything added to a **high-priority** task's loop: new budget.

Messaging

☐	Check
☐	New queue: depth from burst, policy, drop counter, owner of the message memory.

☐	Check
☐	A send or receive with `portMAX_DELAY`: what if it never comes?
☐	A message without a timestamp, or stamped late.
☐	Producer enabled before the consumer exists.
☐	QoS changed: still compatible with every subscriber?

Synchronization

☐	Check
☐	New shared variable: owner, protection, hold time; invariant for tearing.
☐	A lock: mutex (not binary semaphore)? Order consistent? Held across a blocking call? Taken in an ISR?
☐	A call into a driver/library from a high-priority task: does it lock?
☐	`volatile` offered as the fix. (It is not.)

Memory and reliability

☐	Check
☐	New allocation: after init? failure handled? freed? bounded?
☐	New large local array or recursion: stack effect.
☐	New log/file/cache that grows.
☐	A new failure path: what detects it, what reacts, what evidence is kept?
☐	Watchdog feeding changed: still conditioned on progress of every task?

Observability

☐	Check
☐	Does the change add a counter or trace point where it adds risk?
☐	Evidence attached: before/after histogram or trace, not an average.
☐	The observed maximum is not described as WCET.

PART 6 · REFERENCE · SECTION 9

Measuring recipes

Snippets that were run in the labs (RAN) or are standard practice (MANUAL, not executed here). Each answers one question with the least disturbance. Adapt names to your framework.

Time source

Need	Recipe
Microsecond timestamps	a free-running hardware timer: read it, subtract (unsigned). Know its width and wrap. RAN `now_us()` in `labs/rtos/common/bsp.c`
Cycle counts on Cortex-M	`DWT->CYCCNT` after `CoreDebug->DEMCR |= TRCENA` and `DWT->CTRL |= CYCCNTENA`; 32-bit, wraps in 2^{32}/f_cpu. MANUAL (not tested in the emulator)
Zero-overhead period/jitter	toggle a GPIO at the start and end of the job; watch on a logic analyser or oscilloscope. MANUAL

Release, completion, miss counters per periodic task RAN

```
typedef struct { uint32_t releases, completions, misses, max_us; } stat_t;
static void job(stat_t *s, TickType_t release_tick) {
    s->releases++;
    uint32_t t0 = tick_time_us[release_tick % RING];     /* time of the release tick (tick
hook fills it) */
    do_work();
    uint32_t r = now_us() - t0;                           /* response time, incl. waiting
for the CPU */
    s->completions++;
    if (r > s->max_us) s->max_us = r;
    if (r > DEADLINE_US) s->misses++;
}
/* a gap between releases and completions = skipped or running late: count both */
```

Latency histogram (cheap, bounded memory) RAN

```
static uint32_t bucket[16];                /* bucket i counts latencies in [2^i, 2^(i+1))
microseconds */
static void record(uint32_t us) { int i = 0; while (us > 1 && i < 15) { us >>= 1; i++; }
bucket[i]++; }
/* dump the buckets after the test; p99.9 is the bucket where the cumulative count reaches
99.9% */
```

For a precise percentile table dump raw samples and use `labs/tools/lathist.py`.

Context-switch trace RAN

```
/* FreeRTOSConfig.h */
#define traceTASK_SWITCHED_IN()  trace_switch_in_hook()
/* hook: record (time, task number) into a ring buffer; dump on error; render with
labs/tools/trace2gantt.py */
```

Give tasks numbers (`vTaskSetTaskNumber`) so the trace is readable. Keep the buffer in the product; make the scheduler's own error handler dump it (the Pathfinder lesson).

Queue visibility RAN

```
if (xQueueSendToBackFromISR(q, &m, &woken) != pdTRUE) dropped++;     /* count what you
cannot deliver */
uint32_t d = uxQueueMessagesWaitingFromISR(q); if (d > peak) peak = d;
```

Task check-in watchdog RAN

```
/* each task, after a full cycle: */   xEventGroupSetBits(checkins, MY_BIT);
/* supervisor, every N ms (> slowest task period): */
uint32_t bits = xEventGroupClearBits(checkins, ALL_BITS);     /* returns the bits before
clearing */
if ((bits & ALL_BITS) == ALL_BITS) wdt_feed();
else saved_missing = ALL_BITS & ~bits;                          /* record WHO, then do not
feed */
```

Snapshot instead of two reads RAN

```
att_t snap;
taskENTER_CRITICAL();  snap = att;  taskEXIT_CRITICAL();       /* a few instructions: copy
once */
/* use only snap from here on */
```

Stack and heap margins RAN

```
uxTaskGetStackHighWaterMark(handle);           /* words free at the worst point so far */
xPortGetMinimumEverFreeHeapSize();             /* the margin you actually had */
vPortGetHeapStats(&stats);                     /* largest free block, number of free
blocks */
```

CPU share per task RAN

Set `configGENERATE_RUN_TIME_STATS 1`, provide a counter 10–100× faster than the tick, and read `uxTaskGetSystemState()`. Interrupt time is charged to the task it interrupted.

gdb recipes (labs/rtos/gdb) RAN

```
source labs/rtos/gdb/freertos.py                        # fr-tasks, fr-tick, fr-queue
break xTaskDelayUntil if $_caller_is("bus_dist") && H_jobs > 0     # LOGPOINT: log and
continue
commands
  silent
  printf "tick %4d: job %d, response %d ms\n", xTickCount, H_jobs-1, H_resp[H_jobs-1]
  continue
end
watch resets if resets == 1                             # hardware watchpoint on a shared
variable
break xTaskPriorityInherit                              # stop where the kernel boosts a
priority
fr-tasks                                                # who runs, who waits on what, any
inherited priority
fr-queue lock                                           # mutex holder? waiters?
```

Run the target paused with `python3 labs/rtos/run.py ELF --gdb` and attach with `arm-none-eabi-gdb ELF -ex "target remote :1234"`. Build at `-O0` so variables are visible. VS Code: `labs/rtos/gdb/launch.json.example` MANUAL.

> **TRAP** **Observer effect.** A breakpoint stop moved our emulated clock by about 1 ms. On real hardware halted timers may keep running or stop; watchdogs may be frozen. Use logpoints only on rarely-called code, prefer watchpoints, counters and trace, and read results after the run.

PART 6 · REFERENCE · SECTION 10

The implicit-requirements checklist

Forty-five requirements nobody writes down. For a component you own or review, mark each **K** known (evidence in hand), **A** assumed, or **?** unknown. The number of A and ? marks is the size of your risk. Explanations are in book chapter 23.

ID	Requirement	K / A / ?	ID	Requirement	K / A / ?
T1	period and deadline stated		Y1	owner + protection + hold time per datum	
T2	budget is an upper bound		Y2	multi-field data read as a snapshot	
T3	miss policy defined		Y3	mutex (inheritance) for exclusion	
T4	jitter bounded		Y4	blocking time bounded, in R	
T5	interrupts + tick in budget		Y5	lock order; no blocking in a lock	
T6	counters wrap; known		Y6	library/driver locks known	
T7	CPU headroom; exact R		Y7	ISRs: ISR-safe calls, short sections	
T8	rate justified; harmonic		R1	watchdog fed after *every* task progresses	
T9	absolute delays for periods		R2	reaction to failure defined	
S1	priorities follow timing		R3	containment of failures	
S2	no spin; no unbounded wait		R4	restart tested, incl. repeated	
S3	equal-priority sharing OK		R5	evidence survives a reset	
S4	timer callbacks short		R6	stack/heap margins measured	

ID	Requirement	K / A / ?	ID	Requirement	K / A / ?
S5	idle task can run		R7	growth with time bounded	
S6	overload degrades predictably		R8	silent consumers/sensors detected	
M1	queue depth justified		O1	one number + method + margin	
M2	overflow policy named		O2	histogram, open-loop	
M3	drops counted and visible		O3	releases / completions / misses	
M4	age limit + source timestamp		O4	trace left in the product	
M5	rate-crossing rule		O5	observation does not perturb	
M6	QoS matches need		O6	observed max ≠ WCET	
M7	start-up order + readiness		O7	tests drive the worst case	
M8	message ownership clear			**Score:** K __ A __ ? __	

PART 6 · REFERENCE · SECTION 11

Incident patterns

Real failures, reduced to the pattern and the check that would have caught them. Facts and caveats: `research/08-case-files.md`. Sources named so you can read the original.

Incident	Pattern	The check	Source
Apollo 11 (1969) 1201/1202 alarms	unfinished jobs hold a fixed pool; overload accumulates; restart sheds load	pool and queue limits; margin of the lowest-priority job; restart protection	Eyles 2004
Patriot, Dhahran (1991)	clock error grows with uptime (24-bit fixed point)	what is each time value multiplied by, and how long does it stay up?	GAO/IMTEC-92-26
Therac-25 (1985–87)	non-atomic test-and-set on shared variables	one atomic operation, or ownership	Leveson & Turner 1993
Ariane 501 (1996)	unprotected conversion; “on exception, halt”; identical redundant units	exception policy; reuse envelope; redundancy shares systematic bugs	Inquiry Board 1996
Mars Pathfinder (1997)	priority inversion through a library lock; no inheritance	mutex with inheritance; B in analysis; a trace in the product	Reeves 1997
Northeast blackout (2003)	alarm process stalled silently; events queued; server failed	consumer heartbeat; queue age; monitor of the monitor	Task Force 2004; The Register
Spirit rover (2004)	resource consumed at boot grows with time → reset loop	boot-time resources bounded; evidence persists; safe mode that avoids the culprit	Reeves & Neilson 2005
Toyota ETCS (testimony, 2013) SEC	watchdog fed from a timer interrupt; stack nearly full; high load	feeding condition; stack margin; bound vs load	Koopman 2014 (plaintiff’s expert)
Cloudflare (2017)	wall clock difference negative at a leap second	monotonic clock for durations	Cloudflare post-mortem
Boeing 787 (2015)	248-day counter overflow, units powered together	wrap time; common-cause	FAA/EASA AD 2015-09-07

Incident	Pattern	The check	Source
Ingenuity Flight 6 (2021)	one dropped frame + position-based timestamps corrupt later data; load caused the drop	stamp at source; check stamp gaps; budget optional load	NASA/JPL blogs
CAN analysis (1994→2007)	a widely used schedulability analysis was wrong	tools *and* measurement; know the model	Davis et al. 2007
Capstone R10 (lab)	silent drops, torn read, wrong priority, slow path in a fast task	counters; snapshot; rate/deadline priority; move slow work down	this project

Cross-reference to the six questions

Question	Incidents
Timing	Patriot, 787, Cloudflare, Ingenuity
Scheduling	Apollo, Pathfinder, capstone
Messaging	Blackout, Ingenuity, capstone
Synchronization	Therac-25, Pathfinder, capstone
Reliability	Ariane, Spirit, Toyota (testimony), Pathfinder recovery
Observability	Pathfinder trace, capstone histograms

PART 6 · REFERENCE · SECTION 12

Quick reference

The numbers and mappings you reach for twice a day. Full tables: book Appendices B and C.

Formulas

What	Formula
Utilization	$U = \Sigma\ C/T$ (include interrupts)
RM guarantee	$U \le n(2^{1/n} - 1)$: 1.00, 0.83, 0.78, 0.76, 0.74, 0.72 … 0.69
Harmonic periods	RM schedulable to U = 1
Response time	$R = C + B + \Sigma_{hp} \lceil (R + J)/T \rceil \cdot C$; schedulable if $R \le D$
Queue depth	⌈(peak rate − service rate) × burst⌉ + 1
Worst latency in a full queue	depth × service time
Counter wrap	2^bits ÷ rate: 32-bit tick @1 kHz = 49.7 days; 16-bit = 65.5 s; 32-bit @25 MHz = 171.8 s
Clock drift	ppm × 10^{-6} × elapsed: 50 ppm ≈ 4.3 s/day
Time compare	`(int32_t)(now - then) >= 0`
Data age (loose)	$\Sigma\ (T_i + R_i)$
Clean test of N trials	failure probability < ~3/N at 95% confidence

Where each thing is called in FreeRTOS

	FreeRTOS	ISR variant
periodic wait	`xTaskDelayUntil`	—
queue	`xQueueSend/Receive`	`…FromISR`
mutex	`xSemaphoreTake/Give` on a mutex	**none**
binary semaphore (signal)	`xSemaphoreTake/Give`	`xSemaphoreGiveFromISR`
notification	`xTaskNotifyGive`	`vTaskNotifyGiveFromISR`
critical section	`taskENTER_CRITICAL`	`taskENTER_CRITICAL_FROM_ISR`

Same idea, other names

Idea	FreeRTOS	Zephyr	F´	cFS	ROS 2
task	task	thread	active component	app task	executor thread
periodic activation	`DelayUntil`, timer	`k_timer`	rate group	scheduler table slot	timer in executor
queue	queue	message queue	async port queue	SB pipe	subscription queue
lock	mutex (inherit)	`k_mutex` (inherit)	guarded port	OS mutex	callback group / lock
lifecycle	—	—	topology	ES start/stop	managed node

(cFS and Zephyr entries: verify in your build's documentation.)

Watchdog designs

☒ fed from the timer ISR · ☒ fed from a low-priority task · ☑ fed by a supervisor only if every task checked in since the last feed (and windowed, if the hardware allows).

Honest limits of this handbook

All lab numbers are from an emulator with a deterministic clock and one FreeRTOS version (V11.3.2); they demonstrate relationships, not your board's speed. Items marked SEC are expert testimony or press reports. `research/09-claims-we-could-not-verify.md` lists everything unconfirmed.